	TP N°3	Enseignante : Arij Abid

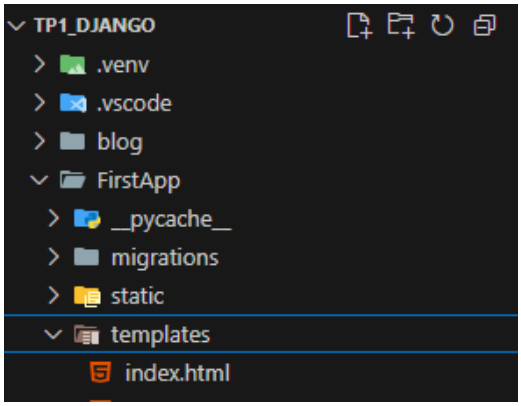
TP3 :

Créer des vues et URLs

1 . Organiser les templates et le CSS :

a. Création du template HTML

- Ajouter un dossier nommé **templates** et un autre sous dossier **FirstApp** dedans. Créer un fichier index.html dans le sous dossier **FirstApp :**



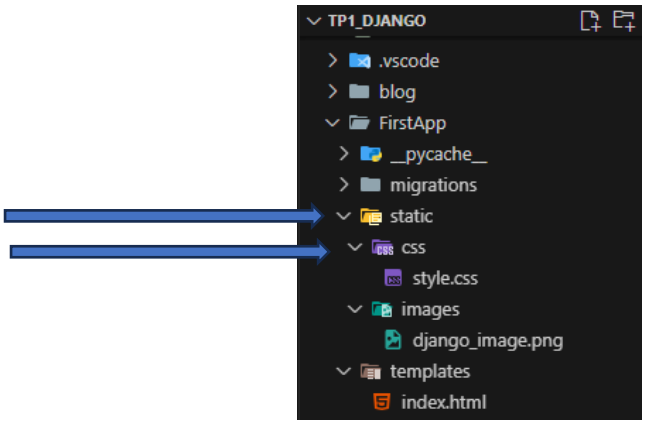
Cette fichier index.html est un **template de base** dans firstApp/templates/index.html

b. Ajouter le CSS (style.css)

Les fichiers statiques

Par défaut Django trouve automatiquement tous les fichiers inclus dans un dossier nommé `static` à l'intérieur d'une application :

Créer un dossier intitulé static, dans ce dossier, créer un autre nommé css ou on va ajouter un fichier style.css



2. Les URLs : relier les pages

Étape 1 : Créer le fichier FirstApp/urls.py

« Créez ce fichier manuellement dans le dossier *firstApp*. »

```
from django.urls import path
from . import  views

urlpatterns = [
   path("index/",views.index, name='index'),
   path("home/",views.home),

]
```

Étape 2 : Relier les URLs de l'application au projet principal

Ouvre `tp1/urls.py` et modifie ainsi :

```
from django.contrib import admin
from django.urls import include, path
from FirstApp import views
urlpatterns = [
   path("admin/", admin.site.urls),
   path("accueil/",views.accueil),
   path("first/",include('FirstApp.urls'))
]
```

❖ Rôle général du include()

La fonction include() sert à lier (ou connecter) **les URLs d'une application à celles du projet principal.**

Autrement dit :

- Le projet Django (par ex. tp1django) gère les URLs principales.
- Chaque application (par ex. FirstApp) a ses propres URLs.
- La fonction include() permet au projet de rediriger les requêtes vers les URLs de cette application.

Contenu de fichier index.html :

- **Téléchargez le code qui se trouve dans le Classroom, ainsi que l'image.**

FirstApp > templates > FirstApp > index.html > .

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Mon site web</title>
    <link rel="stylesheet" href="{% static 'css/style.css' %}">
</head>
<body>
    <div class="container">

        <h2>My First  Application based Django</h2>

    </div>
</body>
</html>
```

❖ Rôle de {% load static %}

Cette ligne permet à **Django** de **charger le système des fichiers statiques**.

Autrement dit, elle dit à Django :

"Je veux utiliser des fichiers statiques (images, CSS, JavaScript) dans ce template."

Sans cette ligne, Django ne reconnaîtra pas la balise :

{% static 'chemin/du/fichier' %}

et tu auras une erreur.

Donc Un **fichier statique** est un fichier qui **ne change pas côté serveur**.
Exemples :

- des fichiers **CSS** (pour le style)
- des fichiers **JavaScript** (pour les interactions)
- des **images** (logo, icônes, photos)

➞ Ces fichiers ne dépendent pas du contenu de la base de données.

Django cherche les fichiers statiques dans un dossier spécial, souvent appelé : /static/

- {% load static %} → active l'accès aux fichiers statiques.
- {% static 'css/style.css' %} → Django remplace ce chemin par l'URL réelle du fichier, par ex. :

<img src="{% static 'images/photo.png' %}" alt="Photo">
=> Django va automatiquement convertir cela en :

<img src="/static/images/photo.png" alt="Photo">

Avantage : ton code marche en développement **et en production**, même si le dossier static change de place.

Balise	Rôle	Exemple	Sert à
{% static %}	Lien vers fichiers statiques	<img src="{% static 'img/logo.png' %}">	Charger images, CSS, JS
{% block %}	Zone personnalisable d'un template	{% block title %}Accueil{% endblock %}	Créer des pages différentes à partir d'un même modèle
{% extends %}	Héritage de template parent	{% extends 'base.html' %}	Réutiliser la structure d'un fichier principal

- Les fichiers **HTML dans Django** sont en réalité des **templates**.
- Ils ne sont **pas statiques** car Django peut les **modifier dynamiquement** avant de les envoyer au navigateur :

- Ici, `{{ user.username }}` sera remplacé par le vrai nom de l'utilisateur connecté.
- Si on mettait ce fichier dans `static`, Django ne pourrait pas faire ce rendu dynamique, car les fichiers statiques sont **servis tels quels**.

```
mon_projet/
|
├─ templates/      <-- fichiers HTML dynamiques
|   └─ index.html
|
├─ static/         <-- fichiers statiques
|   ├─ images/
|   ├─ css/
|   └─ js/
```

- **Templates HTML** = peuvent changer selon les données (`{{ ... }}`) → dossier `templates`
- **Static** = jamais modifiés par Django → dossier `static`

- **Créer plusieurs pages**

Créez un fichier **about.html** qui hérite de **index.html**.
Ensuite, créez un fichier **contact.html** qui hérite également de **index.html**.

Puis modifie le fichier *urls.py* pour ajouter les nouveaux chemins :

```
from django.urls import path
from . import  views

urlpatterns = [
    path("index/",views.index),
    path("home/",views.home),
    path("affiche/",views.affiche, name='homePage'),
    path("about/",views.about, name='aboutPage'),
    path("contact/", views.contact, name='contactPage'),
   # path('liste', views.liste_articles, name='liste_articles'),
]
```

Rôle du name dans une URL Django

Le paramètre name sert à **donner un nom unique à une route (URL)**
➞ Cela permet ensuite d'y faire référence **facilement dans ton code ou dans les templates**, sans devoir écrire à nouveau le chemin complet.

- **Modifier views.py:**

```
def about(request):
    return render(request, 'about.html')

def contact(request):
    return render(request, 'contact.html')
```

- **Créer la page À propos : about.html**

about.html ×
FirstApp > templates > about.html > ...

```
1
2   {% extends 'index.html' %}
3   {% load static %}
4   {% block title %}about{% endblock %}
5
6   {% block content %}
7   <div class="text-center">
8   <h2>À propos de mon projet Django</h2>
9
10  </div>
11  {% endblock %}
```

❖ **Modifier index.html**

```
<li class="nav-item">
  <a class="nav-link" href="{% url 'aboutPage' %}">About</a>
</li>
<li class="nav-item">
  <a class="nav-link" href="{% url 'contactPage' %}">Contact</a>
</li>
```

La balise Django {% url %} sert à générer dynamiquement l'URL d'une route en utilisant le nom défini dans urls.py.
href="{% url 'homePage' %}" : utilise le nom de l'URL défini dans urls.py.

Avantages :

1. Si tu changes l'URL dans urls.py, les liens dans les templates fonctionneront toujours.
2. Permet d'éviter d'écrire l'URL en dur.
3. Rend le projet plus flexible et maintenable.

On va tester les filtres des temples :

Modifier la vue contact pour passer une variable

Dans ton views.py :

```
def contact(request):
    nom_utilisateur = "arij abid"
    emails = ["contact@exemple.com", "support@exemple.com", "info@exemple.com"]
    date_actuelle = datetime.now()

    return render(request, 'contact.html', {
        'nom_utilisateur': nom_utilisateur,
        'emails': emails,
        'date_actuelle': date_actuelle
    })
```

1. **nom_utilisateur = "arij abid"**

- On crée une variable Python appelée nom_utilisateur.
- Elle contient une chaîne de caractères "arij abid".
- Cette variable sera **envoyée au template** pour être affichée.

2. **emails = ["contact@exemple.com", "support@exemple.com", "info@exemple.com"]**

- On crée une **liste Python** appelée emails.
- Elle contient **3 adresses email**.
- Les listes peuvent être parcourues dans le template pour afficher chaque email ou utiliser des filtres comme join ou length.

3. **date_actuelle = datetime.now()**

- datetime.now() → récupère **la date et l'heure actuelles**.
- On stocke cette valeur dans date_actuelle.
- Elle pourra être affichée dans le template et formatée avec le filtre date de Django.

Exemple d'affichage dans le template :
{{ date_actuelle|date:"d/m/Y H:i" }} → affiche 10/11/2025 20:45.

4.return render(request, 'contact.html', { ... })

- render() → fonction Django qui **relie la vue à un template HTML**.
- Paramètres :
    1. request → la requête reçue.
    2. 'contact.html' → le nom du template à utiliser.
    3. `{ ... }` → dictionnaire Python : **on envoie des variables au template**.

    `'nom_utilisateur': nom_utilisateur` → dans le template, tu pourras faire `{{ nom_utilisateur }}`

    `'emails': emails` → tu pourras faire `{{ emails|join:", " }}`

    `'date_actuelle': date_actuelle` → tu pourras faire `{{ date_actuelle|date:"d/m/Y H:i" }}`

En résumé : cette ligne **renvoie au navigateur la page `contact.html` avec toutes les variables prêtes à l'affichage**.

Filtre	Rôle	Exemple
title	Met la première lettre de chaque mot en majuscule	"arij abid" → "Arij Abid"
join:", "	Transforme une liste en chaîne de caractères séparée par une virgule	["a","b","c"] → "a, b, c"
length	Compte le nombre d'éléments dans une liste ou caractères dans une chaîne	["a","b","c"] → 3
date:"format"	**Formate un objet datetime selon le format donné**	**"d/m/Y H:i" → "10/11/2025 20:45"**

- d → jour à 2 chiffres
- m → mois à 2 chiffres
- Y → année sur 4 chiffres
- H → heure 24h
- i → minutes

Filtre	Description
upper	**Met le texte en majuscules**
lower	**Met le texte en minuscules**
title	**Met la première lettre de chaque mot en majuscule**
truncatechars:n	**Limite le texte à n caractères**
date:"format"	**Formate une date selon le format donné**
length	**Retourne le nombre d'éléments dans une liste ou chaîne**
join:","	**Joint les éléments d'une liste avec un séparateur**